

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Вычислительная математика»
Тема: Методы решений нелинейных уравнений

Студент гр. 4341

Малышев К.Д.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Целью лабораторной работы являются изучение методов бисекции, хорд, простых итераций и метода Ньютона для поиска корня уравнения с заданной точностью, написание программы, реализующей данный метод, и исследование скорости сходимости и обусловленности метода (чувствительности к ошибкам в исходных данных).

Задание.

Требуется найти корень уравнения $f(x) = 0$ указанными методами с заданной точностью Eps , исследовать скорость сходимости и обусловленность метода (чувствительность к ошибкам в исходных данных). Порядок выполнения работы должен быть следующим:

1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (найти отрезки $[Left, Right]$, на которых функция $f(x)$ удовлетворяет условиям теоремы Коши).

2) Составить подпрограмму вычисления функции $f(x)$.

3) Для метода бисекций:

1. Составить подпрограмму вычисления функции $f'(x)$, для вычисления v_{Δ} .
2. Составить главную программу, содержащую обращение к подпрограммам $f(x)$, $f'(x)$, BISECT, Round и вывод результатов.
3. Провести вычисления по программе. Построить график зависимости числа итераций k от Eps .
4. Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы Round, округляющей значения функции с заданной точностью Delta.

4) Для метода хорд:

1. Составить подпрограмму - функцию вычисления функции $f(x)$, предусмотрев округление значений функции с заданной точностью Delta с использованием программы Round.
2. Составить главную программу, вычисляющую корень уравнения $f(x) = 0$ и содержащую обращение к подпрограмме $f(x)$, HORDA, Round и индикацию результатов.
3. Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

5) Для метода простых итераций:

1. Преобразовать уравнение $f(x) = 0$ к виду $x = \phi(x)$ так, чтобы в некоторой окрестности $[Left, Right]$ корня производная $\phi'(x)$ удовлетворяла условию $|\phi'(x)| \leq q < 1$. При этом следует иметь в виду, что чем меньше величина q , тем быстрее последовательные приближения сходятся к корню.
2. Выбрать начальное приближение, лежащее на $[Left, Right]$.
3. Составить подпрограмму для вычисления значений $\phi(x)$, $\phi'(x)$ предусмотрев округление вычисленных значений с точностью Delta.
4. Составить главную программу, вычисляющую корень уравнения и содержащую обращение к программам $\phi(x)$, $\phi'(x)$ и ITER и индикацию результатов.
5. Провести вычисления по программе. Исследовать скорость сходимости и обусловленность метода.

6) Для метода Ньютона:

1. Составить подпрограммы - функции вычисления $f(x)$, $f'(x)$ предусмотрев округление их значений с заданной точностью Delta.
2. Составить главную программу, вычисляющую корень уравнения $f(x) = 0$ и содержащую обращение к подпрограммам $f(x)$, $f'(x)$, Round, NEWTON и индикацию результатов.
3. Выбрать начальное приближение корня x_0 из [Left, Right] так, чтобы $f(x) \cdot f''(x) > 0$!!!!!!
4. Провести вычисления по программе. Исследовать скорость сходимости метода и чувствительность метода к ошибкам в исходных данных.

Вариант 12: $f(x) = (1 + \cos(x)) / (3 - \sin(x)) - x$;

Отделение корней уравнения.

Рассмотрим функцию $f(x) = (1 + \cos(x)) / (3 - \sin(x)) - x$;

Функция состоит из тригонометрических функций, которые определены и непрерывны на всей числовой прямой. Знаменатель $3 - \sin x$ не обращается в ноль, так как $-1 \leq \sin x \leq 1$, следовательно $2 \leq 3 - \sin x \leq 4$

Поэтому функция $f(x)$ непрерывна на интервале $(-\infty, +\infty)$.

Найдем производную $f'(x)$:

Используем правило производной частного $(u / v)' = (u'v - uv') / v^2$

$$u = 1 + \cos x \Rightarrow u' = -\sin x$$

$$v = 3 - \sin x \Rightarrow v' = -\cos x$$

$$f'(x) = ((-\sin x)(3 - \sin x) - (1 + \cos x)(-\cos x)) / (3 - \sin x)^2$$

Упростим числитель:

$$(-\sin x)(3 - \sin x) = -3\sin x + \sin^2 x$$

$$-(1 + \cos x)(-\cos x) = \cos x + \cos^2 x$$

$$-3\sin x + \sin^2 x + \cos x + \cos^2 x = 1 - 3\sin x + \cos x$$

Не забываем, что $(-x)' = -1$

$$f'(x) = ((1 - 3\sin x + \cos x) / (3 - \sin x)^2) - 1$$

$(3 - \sin x)^2 > 0$ при любых $x \Rightarrow$ знак дроби определяется только числителем

Приведем к одной дроби:

$$f'(x) = ((1 - 3\sin x + \cos x - (3 - \sin x)^2) / (3 - \sin x)^2)$$

$$(3 - \sin x)^2 = 9 - 6\sin x + \sin^2 x$$

$$1 - 3\sin x + \cos x - 9 + 6\sin x - \sin^2 x = -8 + 3\sin x + \cos x - \sin^2 x$$

Оценим максимально возможное значение:

- $\sin x \leq 1$
- $\cos x \leq 1$
- $\sin^2 x \geq 0$

Берем максимальные значения и подставляем:

$$-8 + 3 + 1 - 0 = -4 < 0 \Rightarrow f'(x) < 0$$

Следовательно функция монотонно убывает, поэтому уравнение может иметь не более одного корня

Локализуем корень.

при $x = 0$:

$$f(0) = 0,333$$

при $x = 1$:

$$f(1) = -0,198$$

Следовательно $f(1) < 0, f(0) > 0$

Так как функция непрерывна на $[0,1]$ и принимает значения разных знаков на концах интервала, то по теореме Больцано на этом интервале существует корень.

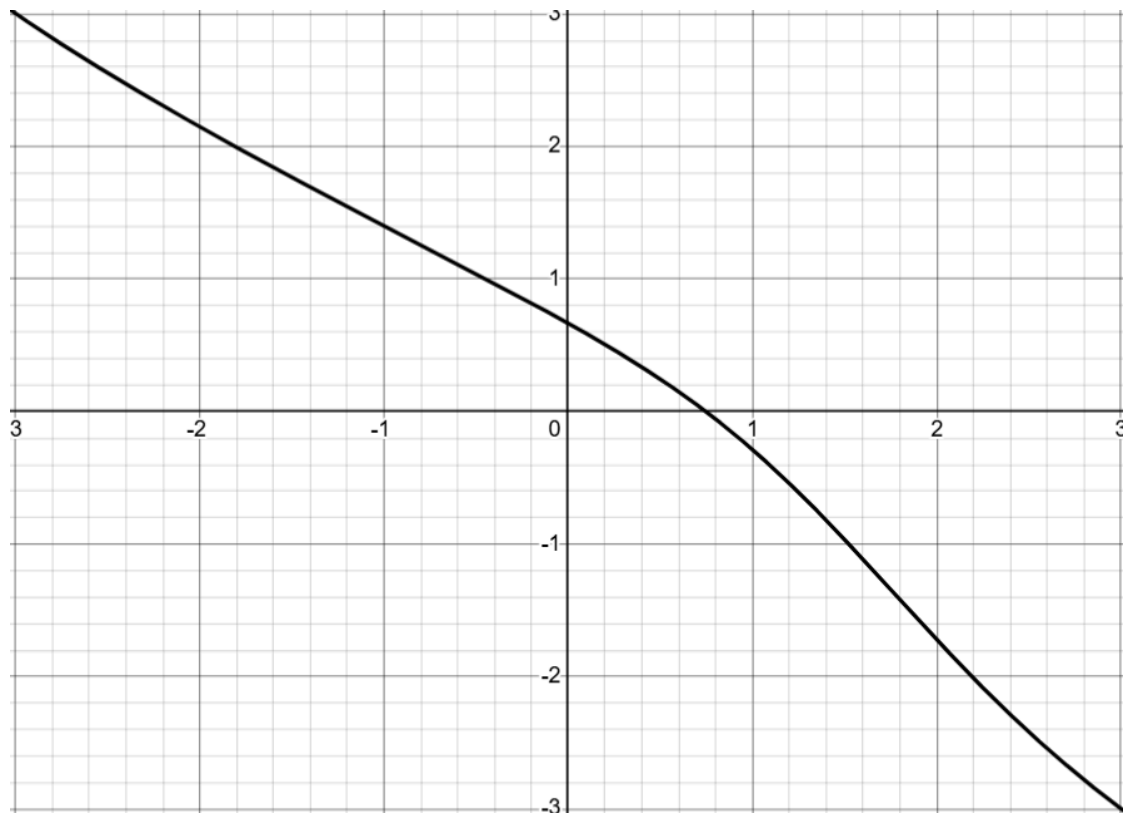


Рисунок 1 - График $f(x)$

Исследование производной на отделимость от 0.

Приравниваем $f'(x) = 0$:

$$((1 - 3\sin x + \cos x) / (3 - \sin x)^2) = 1$$

$$1 - 3\sin x + \cos x = (3 - \sin x)^2$$

$$8 - 3\sin x - \cos x + \sin^2 x = 0$$

Оценим снизу (минимум):

- $\sin x \in [-1, 1]$
- $\cos x \in [-1, 1]$
- $\sin^2 x \geq 0$

Берём наихудший случай (чтобы сделать выражение как можно меньше):

$$8 - 3 + 0 - 1 = 4 \Rightarrow 4 > 0$$

Следовательно выражение больше 0 при любом значении.

Преобразование уравнения к виду $x = \varphi(x)$.

Для применения метода простых итераций необходимо преобразовать исходное уравнение к виду $x = \varphi(x)$ так, чтобы на отрезке $[Left, Right]$ выполнялось условие $|\varphi'(x)| \leq q < 1$.

При этом чем меньше число q , тем быстрее сходятся последовательные приближения.

Исходное уравнение $(1 + \cos(x)) / (3 - \sin(x)) - x = 0$ преобразуем к виду $(1 + \cos(x)) / (3 - \sin(x)) = x$

Тогда $\varphi(x) = (1 + \cos(x)) / (3 - \sin(x))$, а её производная равна $\varphi'(x) = -\sin x \cdot (3 - \sin x) - (1 + \cos x)(-\cos x) / (3 - \sin x)^2$.

Упростим числитель:

$$-\sin x \cdot (3 - \sin x) - (1 + \cos x)(-\cos x) = -3\sin x + \sin^2 x + \cos x + \cos^2 x = 1 - 3\sin x + \cos x$$

На отрезке $[0, 1]$ имеем

$$\cos(0) = 1 \quad \cos(1) = 0.54$$

$$\sin(0) = 0 \quad \sin(1) = 0.84$$

$$\varphi'(0) = 1 - 0 + 1/9 = 2/9 = 0.2222$$

$$\varphi'(1) = 1 - 2.52 + 0.54 / 4.66 = -0.98 / 4.66 = -0.21$$

Следовательно, на всём отрезке $[0,1]$ выполняется оценка $|\varphi'(x)| \leq q < 1$. где можно взять $q = 0.23$

Таким образом, функция $\varphi(x) = (1 + \cos(x)) / (3 - \sin(x))$, удовлетворяет условию сходимости метода простых итераций на отрезке $[0,1]$.

Поскольку на данном отрезке $\varphi'(x)$ меняет знак, последовательные приближения будут сходиться к корню с колебаниями около него, что соответствует теоретическим свойствам метода простых итераций.

Разработка программы.

Для решения уравнения $f(x) = 0$ был реализован алгоритм метода бисекции. Программа написана на языке C++ и состоит из нескольких подпрограмм, каждая из которых выполняет определённую функцию.

Сначала была реализована подпрограмма вычисления значения функции $f(x)$:

```
double F(double x) {  
    return (1.0 + cos(x)) / (3.0 - sin(x)) - x;  
}
```

Для анализа погрешности вычислений также была реализована подпрограмма вычисления производной функции:

```
double dF(double x) {  
    double numerator =  
        (-sin(x)) * (3.0 - sin(x)) -  
        (1.0 + cos(x)) * (-cos(x));  
  
    double denominator =  
        (3.0 - sin(x)) * (3.0 - sin(x));  
  
    return numerator / denominator - 1.0;  
}
```

Для моделирования ошибок вычислений была разработана вспомогательная функция Round, которая выполняет округление значения функции с заданной точностью Δ .

```
double Round(double X, double Delta) {  
    if (Delta <= 0) return X;  
    if (X > 0.0) return Delta * (long((X / Delta) + 0.5));  
    else return Delta * (long((X / Delta) - 0.5));  
}
```

Основной алгоритм поиска корня реализован в подпрограмме BISECT, которая выполняет метод бисекции.

Метод заключается в последовательном делении отрезка пополам и выборе той половины интервала, на которой функция меняет знак.

```
double BISECT(double Left, double Right, double Eps, int &N)
```

Подпрограмма принимает:

- левую границу интервала Left
- правую границу интервала Right
- требуемую точность Eps
- переменную N, в которой возвращается число итераций.

Алгоритм завершает работу, когда длина интервала становится меньше $2 \cdot \text{Eps}$. В качестве приближённого значения корня берётся середина полученного интервала.

Для исследования влияния ошибок исходных данных была реализована модифицированная версия метода бисекции BISECT_Rounded. В этой подпрограмме значения функции округляются с помощью функции Round.

Главная программа выполняет следующие действия:

1. задаёт начальный интервал поиска корня;
2. вызывает метод бисекции для различных значений точности Eps;
3. выводит найденный корень и число итераций;
4. вычисляет значение функции и её производной в найденной точке;
5. выполняет исследование зависимости числа итераций от точности;
6. исследует чувствительность метода к ошибкам вычисления функции.

Программа, реализующая метод хорд, была написана на языке программирования C++.

Была написана функция `double F(double x)` для вычисления значения функции $f(x)$:

```
double F(double x) {  
    return (1.0 + cos(x)) / (3.0 - sin(x)) - x;  
}
```

Была написана функция `double dF(double x)` для вычисления производной функции $f(x)$:

```
double dF(double x) {  
  
    double numerator =  
        (-sin(x)) * (3.0 - sin(x)) -  
        (1.0 + cos(x)) * (-cos(x));  
  
    double denominator =  
        (3.0 - sin(x)) * (3.0 - sin(x));  
  
    return numerator / denominator - 1.0;  
}
```

Функция `double HORDA(double Left, double Right, double Eps, int &N)` реализует метод хорд для нахождения корня функции.

Метод хорд итеративно уточняет значение корня, используя линейную интерполяцию между двумя точками на отрезке $[Left, Right]$, пока значение функции в текущем приближении не станет меньше заданной точности `Eps`.

Принимает на вход:

- `Left` — левая граница интервала, на котором локализован корень

- Right — правая граница интервала локализации корня
- Eps — требуемая точность вычисления корня
- N — количество выполненных итераций метода.

Также была реализована функция `double Round(double X, double Delta)` которая возвращает значение X, округлённое с точностью Δ.

Принимает на вход:

- X — число, которое необходимо округлить
- Delta — точность округления.

Округление используется для моделирования погрешностей вычислений и исследования влияния округления на скорость сходимости метода.

Программа, реализующая метод простых итераций, была написана на языке программирования C++.

Была написана функция `double f(double x)` для вычисления значения функции $f(x) = (1 + \cos(x)) / (3 - \sin(x)) - x$

Также была реализована функция `double phi(double x)` для вычисления значения итерационной функции

$$\varphi(x) = (1 + \cos(x)) / (3 - \sin(x)),$$

Кроме того, была реализована функция `double dphi(double x)` для

вычисления производной итерационной функции:

$$\varphi'(x) = 1 - 3\sin x + \cos x / (3 - \sin x)^2.$$

Для исследования чувствительности метода к ошибкам в исходных данных была реализована функция `double Round(double x,`

`double delta`), возвращающая значение `xxx`, округленное с точностью `delta`.

Функция принимает на вход:

- `x` — значение для округления;
- `delta` — точность округления.

Также была реализована функция `double phi_round(double x, double delta)`, возвращающая округлённое значение функции $\varphi(x)$.

Функция `double ITER(double X0, double Eps, int& N)` реализует метод простых итераций для нахождения корня уравнения. На каждой итерации новое приближение вычисляется по формуле

$$x_{n+1} = \varphi(x_n).$$

Итерационный процесс продолжается до тех пор, пока не будет достигнута заданная точность `Eps`.

Функция принимает на вход:

- `X0` — начальное приближение корня;
- `Eps` — требуемая точность вычисления корня;
- `N` — количество итераций.

В программе также реализовано исследование скорости сходимости метода, а также исследование чувствительности к погрешностям округления входных данных.

Разработанный программный код приведен в приложении А.

Программа, реализующая метод Ньютона, была написана на языке программирования C++.

Была написана функция `double F(double x)` для вычисления значения функции

$$f(x) = (1 + \cos(x)) / (3 - \sin(x)) - x;$$

Также были реализованы функции `double dF(double x)` и `double ddF(double x)` для вычисления первой и второй производных:

$$f'(x) = ((1 - 3\sin x + \cos x) / (3 - \sin x)^2) - 1$$

$$f''(x) = ((-3\cos x - \sin x)(3 - \sin x)^2 + 2(1 - 3\sin x + \cos x)(3 - \sin x)\cos x) / (3 - \sin x)^4$$

Функция `double newton(double x0, double eps, std::vector<double>& steps)` реализует метод Ньютона для нахождения корня уравнения. На каждой итерации новое приближение вычисляется по формуле

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Итерационный процесс продолжается до тех пор, пока значение функции в текущем приближении не станет меньше заданной точности `eps`.

Функция принимает на вход:

- `x0` — начальное приближение корня;
- `eps` — требуемая точность вычисления корня;
- `N` — количество итераций (передается по ссылке).

Также была реализована функция `double Round(double x, double delta)`, возвращающая значение `x`, округлённое с точностью `delta`.

Функция принимает на вход:

- `x` — значение для округления;
- `delta` — точность округления.

Для исследования чувствительности метода к ошибкам в исходных данных была реализована функция `double F_round(double x,`

`double delta)`, возвращающая округлённое значение функции. Кроме того, в главной программе реализован метод Ньютона с применением округления, где на каждой итерации значения функции и её производной округляются с точностью `Delta`. Это позволяет исследовать чувствительность метода к погрешностям исходных данных.

Разработанный программный код приведён в приложении А.

Проведение вычислений и построение графика для метода бисекции.

Для исследования работы метода бисекции были проведены вычисления при различных значениях точности `Eps`.

Точность изменялась в диапазоне:

$$Eps = 10^{-2}, 10^{-3}, 10^{-4}, \dots, 10^{-12}$$

Для каждого значения точности программа вычисляла:

- приближённое значение корня уравнения;
- значение функции в найденной точке;
- значение производной;
- число итераций k , необходимых для достижения заданной точности.

Eps	Root x	N	f(x)	f'(x)	Delta_teor
1e-02	0.7421875000	6	5.20e-03	-1.0538	4.93e-03
1e-03	0.7470703125	9	4.32e-05	-1.0566	4.09e-05
1e-04	0.7471313477	13	-2.13e-05	-1.0566	2.02e-05
1e-05	0.7471084595	16	2.89e-06	-1.0566	2.74e-06
1e-06	0.7471113205	19	-1.32e-07	-1.0566	1.25e-07
1e-07	0.7471111417	23	5.70e-08	-1.0566	5.40e-08
1e-08	0.7471111938	26	1.92e-09	-1.0566	1.82e-09
1e-09	0.7471111948	29	9.41e-10	-1.0566	8.90e-10
1e-10	0.7471111956	33	1.81e-11	-1.0566	1.72e-11
1e-11	0.7471111957	36	-4.93e-12	-1.0566	4.67e-12
1e-12	0.7471111957	39	-1.25e-13	-1.0566	1.18e-13

Таблица 1

Анализ полученных данных показывает, что при уменьшении значения Eps требуемая точность возрастает, вследствие чего увеличивается и число итераций метода бисекции.

Это объясняется тем, что на каждой итерации метода длина интервала уменьшается в два раза. Поэтому для достижения более высокой точности требуется большее количество шагов алгоритма.

На основании полученных данных был построен график зависимости числа итераций k от точности Eps.

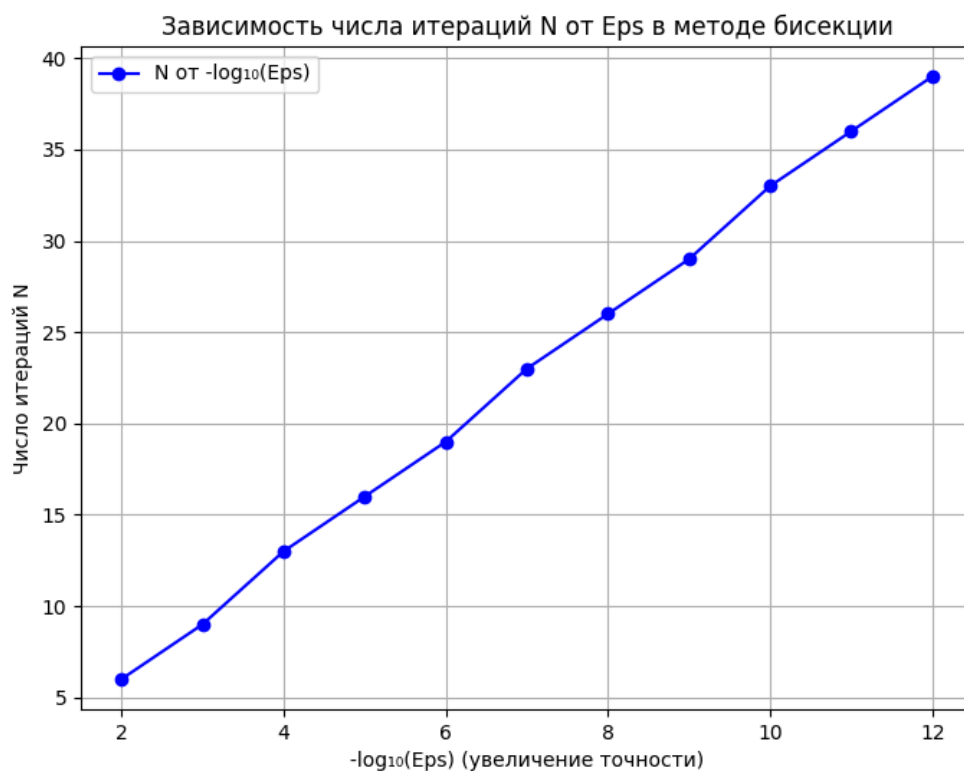


Рисунок 1. Зависимость N от Eps

На графике видно, что при уменьшении значения Eps число итераций монотонно возрастает.

Исследование чувствительности метода к ошибкам в исходных данных в методе бисекций.

Для исследования чувствительности метода бисекции к ошибкам в исходных данных была проведена серия вычислений с искусственно введёнными погрешностями при вычислении функции.

Ошибки моделировались с помощью подпрограммы Round, которая выполняет округление значения функции с заданной точностью Δ . Таким образом имитируется ситуация, когда значение функции вычисляется с ограниченной точностью.

В программе была реализована модифицированная версия метода бисекции, в которой значения функции перед использованием в алгоритме округляются: $f(x) \rightarrow \text{Round}(f(x), \Delta)$

Исследование проводилось при фиксированной точности метода $\text{Eps} = 10^{-12}$ и различные значения параметра округления $\Delta = 10^{-1}, 10^{-2}, 10^{-3}, \dots, 10^{-8}$

Для каждого значения Δ определялись:

1. найденное приближённое значение корня;
2. число итераций метода;
3. истинное значение функции в найденной точке (без округления).

Результаты вычислений выводились программой в виде таблицы.

Delta	Root x	N	actual f(root)
1e-01	0.7500000000	1	-3.05477e-03
1e-02	0.7500000000	1	-3.05477e-03
1e-03	0.7470703125	9	4.31977e-05
1e-04	0.7470703125	9	4.31977e-05
1e-05	0.7471084595	16	2.89112e-06
1e-06	0.7471113205	19	-1.31907e-07
1e-07	0.7471112013	22	-5.94773e-09
1e-08	0.7471111938	26	1.92475e-09
0e+00	0.7471111957	39	-1.25122e-13

Таблица 2

Анализ полученных результатов показывает, что при увеличении величины округления Δ погрешность вычисления функции возрастает, что может приводить к изменению найденного приближённого значения корня.

При уменьшении значения Δ влияние ошибок уменьшается, и результат метода приближается к значению, полученному без округления.

Таким образом, метод бисекции является достаточно устойчивым к небольшим ошибкам в вычислении функции, однако при значительных погрешностях исходных данных точность определения корня может ухудшаться.

Исследование скорости сходимости в методе хорд.

Для вычисления ошибки и скорости сходимости в *horda* возвращался значение x каждой итерации. Скорость сходимости для метода хорд

линейная и вычисляется по формуле $C \approx \frac{|x_{n+1} - x^*|}{|x_n - x^*|}$

Исследование скорости сходимости метода хорд			
Итерации	Корень	Ошибка	C сходимости
1	0.6994882025	6.995e-01	0.00000
2	0.7439093842	4.442e-02	0.00000
3	0.7468964866	2.987e-03	0.06725
4	0.7470968005	2.003e-04	0.06706
5	0.7471102305	1.343e-05	0.06704
6	0.7471111310	9.004e-07	0.06705
7	0.7471111310	0.000e+00	0.00000
8	0.7471111913	6.031e-08	0.00000
9	0.7471111954	4.062e-09	0.06735
10	0.7471111956	2.649e-10	0.06522
11	0.7471111956	0.000e+00	0.00000
12	0.7471111956	0.000e+00	0.00000
13	0.7471111956	0.000e+00	0.00000
14	0.7471111956	0.000e+00	0.00000
15	0.7471111956	0.000e+00	0.00000
16	0.7471111956	0.000e+00	0.00000
17	0.7471111956	0.000e+00	0.00000
18	0.7471111956	0.000e+00	0.00000
19	0.7471111956	0.000e+00	0.00000
20	0.7471111956	0.000e+00	0.00000

Полученные результаты вычислений показали, что после нескольких итераций коэффициент сходимости стабилизируется и принимает значение приблизительно $C \approx 0.067$

Это свидетельствует о линейной скорости сходимости метода хорд.

После 8–10 итераций значение корня практически перестаёт изменяться и достигает значения $x \approx 0.7471111956$

Таким образом, проведённое исследование показывает, что метод хорд сходится достаточно быстро и обеспечивает получение корня с высокой точностью.

Тестирование метода хорд.

В табл. 1 приведены результаты работы программы для различных значений эпсилон.

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные
1.	0.1	0.699488
2.	0.01	0.743909
3.	0.000001	0.747111

Чувствительность метода хорд к ошибкам в исходных данных.

Для анализа чувствительности метода к ошибкам в исходных данных программа была дополнена функцией $\text{Round}(x, \text{delta})$, которая выполняет округление значений функции $f(x)$ с заданной точностью delta . Округление используется для моделирования ошибок вычислений и исследования влияния погрешностей исходных данных на результат работы метода хорд.

Для каждого значения точности Eps последовательно перебирались различные значения параметра округления Δ , чтобы определить влияние ошибок исходных данных на найденное значение корня. В ходе исследования использовались значения

$Eps=10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}$

и

$\Delta=10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}$.

В качестве эталонного значения было принято значение корня $x \approx 0.7471111956$ полученное без моделирования ошибок округления при высокой точности вычислений.

Таблица 2 – Исследование чувствительности метода к ошибкам в данных

Eps = 1e-1		
Delta	Root	Error
1e-1	0.6994882025	4.762e-02
1e-2	0.6994882025	4.762e-02
1e-3	0.6994882025	4.762e-02
1e-4	0.6994882025	4.762e-02
1e-5	0.6994882025	4.762e-02
1e-6	0.6994882025	4.762e-02

Eps = 1e-2		
Delta	Root	Error
1e-1	0.6994882025	4.762e-02
1e-2	0.7441525701	2.959e-03
1e-3	0.7441525701	2.959e-03
1e-4	0.7439242104	3.187e-03
1e-5	0.7439089720	3.202e-03
1e-6	0.7439097339	3.201e-03

Eps = 1e-3		
Delta	Root	Error
1e-1	0.6994882025	4.762e-02
1e-2	0.7441525701	2.959e-03
1e-3	0.7468046531	3.065e-04
1e-4	0.7469284350	1.828e-04
1e-5	0.7468959085	2.153e-04
1e-6	0.7468966616	2.145e-04

Eps = 1e-4		
Delta	Root	Error
1e-1	0.6994882025	4.762e-02
1e-2	0.7441525701	2.959e-03
1e-3	0.7468046531	3.065e-04
1e-4	0.7471050308	6.165e-06
1e-5	0.7470989985	1.220e-05
1e-6	0.7470971040	1.409e-05

Eps = 1e-5		
Delta	Root	Error
1e-1	0.6994882025	4.762e-02
1e-2	0.7441525701	2.959e-03
1e-3	0.7468046531	3.065e-04
1e-4	0.7471050308	6.165e-06
1e-5	0.7471078282	3.367e-06
1e-6	0.7471103484	8.472e-07

Eps = 1e-6		
Delta	Root	Error
1e-1	0.6994882025	4.762e-02
1e-2	0.7441525701	2.959e-03
1e-3	0.7468046531	3.065e-04
1e-4	0.7471050308	6.165e-06
1e-5	0.7471078282	3.367e-06
1e-6	0.7471112314	3.578e-08

Из результатов таблицы видно, что при уменьшении величины Δ уменьшается и ошибка в вычисленном значении корня. При относительно больших значениях параметра округления результат заметно отличается от эталонного значения, однако при уменьшении Δ найденное значение корня быстро приближается к точному.

Можно заметить, что при небольших ошибках в исходных данных ошибка в результатах метода остаётся сравнительно малой:

при $\Delta=10^{-3}$ погрешность проявляется примерно в четвёртом знаке после запятой,

при $\Delta=10^{-4}$ — в пятом знаке,

при $\Delta=10^{-5}$ — в шестом знаке.

Таким образом, проведённое исследование показывает, что метод хорд является достаточно устойчивым к ошибкам во входных данных и позволяет получать точные результаты даже при наличии округления.

Исследование скорости сходимости в методе Ньютона.

После выбора начального приближения $x_0 = 1$ были проведены вычисления по программе методом Ньютона.

Для получения эталонного значения корня использовался метод Ньютона с высокой точностью вычислений. В результате было получено эталонное значение корня:

$$x^* = 0.747111195657.$$

Для исследования скорости сходимости в функции NEWTON сохранялись все последовательные приближения x_n , полученные на каждой итерации.

Метод Ньютона обладает квадратической сходимостью. Это означает, что при выполнении условий применимости метода и при удачном выборе начального приближения ошибка очередного приближения убывает очень быстро и пропорциональна квадрату ошибки предыдущего приближения.

Для каждой итерации вычислялась абсолютная ошибка

$$\Delta_n = |x_n - x^*|,$$

где x^* — эталонное значение корня.

Полученные результаты приведены в таблице.

Итерация	Корень	Ошибка
1	0.763534361512	0.016423165855
2	0.747183991387	0.000072795729
3	0.747111197089	0.000000001431
4	0.747111195657	0.000000000000
5	0.747111195657	0.000000000000

Из таблицы видно, что с каждой итерацией ошибка резко уменьшается. Уже на четвертой итерации достигается очень высокая точность. Это подтверждает высокую скорость сходимости метода Ньютона и его квадратический характер сходимости.

Тестирование метода Ньютона.

В табл. 1 приведены результаты работы программы для различных значений эпсилон.

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные
1	0.1	0.747183991387
2	0.01	0.747111197089
3	0.001	0.747111197089
4	0.0001	0.747111197089
5	0.00001	0.747111195657
6	0.000001	0.747111195657

Из результатов тестирования видно, что при уменьшении значения Eps найденное приближение уточняется и приближается к эталонному значению корня. При значении

$$Eps = 10^{-5}$$

полученный результат совпадает с эталонным корнем.

Чувствительность метода Ньютона к ошибкам в исходных данных.

Для анализа чувствительности метода Ньютона к ошибкам в исходных данных код был дополнен функцией Round, которая округляла значения функции и производных с заданной точностью Delta. Это

позволяло моделировать влияние ошибок исходных данных на результат вычислений.

Для каждого значения Eps перебиралось несколько значений $Delta$, после чего определялось приближённое значение корня методом Ньютона. Полученные результаты приведены в табл. 3.

Таблица 3 – Исследование чувствительности метода к ошибкам в данных

Eps	Delta	Результаты метода
0.1	0.1	0.75000000000000
0.1	0.01	0.760330578512
0.1	0.001	0.763831544178
0.1	0.0001	0.763540290621
0.1	0.00001	0.763535938971
0.1	0.000001	0.763534527669
0.01	0.1	0.75000000000000
0.01	0.01	0.760330578512
0.01	0.001	0.746945990707
0.01	0.0001	0.747219119999
0.01	0.00001	0.747186014951
0.01	0.000001	0.747183634941
0.001	0.1	0.75000000000000
0.001	0.01	0.750896616248
0.001	0.001	0.746945990707
0.001	0.0001	0.747219119999
0.001	0.00001	0.747186014951
0.001	0.000001	0.747183634941
0.0001	0.1	0.75000000000000
0.0001	0.01	0.750896616248
0.0001	0.001	0.746945990707
0.0001	0.0001	0.747219119999
0.0001	0.00001	0.747186014951
0.0001	0.000001	0.747183634941
0.00001	0.1	0.75000000000000

0.00001	0.01	0.750896616248
0.00001	0.001	0.746945990707
0.00001	0.0001	0.747124485760
0.00001	0.00001	0.747110305411
0.00001	0.000001	0.747110764371
0.000001	0.1	0.750000000000
0.000001	0.01	0.750896616248
0.000001	0.001	0.746945990707
0.000001	0.0001	0.747124485760
0.000001	0.00001	0.747110305411
0.000001	0.000001	0.747110764371

Исследование скорости сходимости метода простых итераций.

После выбора начального приближения $x_0 = 1$ были проведены вычисления методом простых итераций.

Для получения эталонного значения корня использовался метод с высокой точностью вычислений. В результате было получено:

$$x^* = 0.747111195657.$$

Количество итераций, необходимое для получения эталонного значения, составило 13.

Метод простых итераций обладает линейной сходимостью. Это означает, что ошибка очередного приближения уменьшается примерно пропорционально ошибке предыдущего приближения.

Для каждой итерации вычислялась абсолютная ошибка:

$$\Delta_n = |x_n - x^*|.$$

Результаты вычислений приведены в таблице.

Таблица 1 – Исследование скорости сходимости

Итерация	Значение (x_n)	Ошибка
1	1.000000000000	0.252888804343
2	0.713588881607	0.033522314050
3	0.748691538955	0.001580343298
4	0.747020993431	0.000090202227
5	0.747116301166	0.000005105509
6	0.747110906543	0.000000289114
7	0.747111212029	0.000000016371
8	0.747111194730	0.000000000927
9	0.747111195710	0.000000000052
10	0.747111195654	0.000000000003

Из таблицы видно, что последовательность приближений сходится к корню с постепенным уменьшением ошибки. Сходимость происходит с колебаниями относительно корня, что объясняется сменой знака производной $\varphi'(x)$. Характер уменьшения ошибки подтверждает линейную скорость сходимости метода.

Тестирование метода простых итераций.

В табл. 1 приведены результаты работы программы для различных значений эпсилон.

Таблица 1 – Результаты тестирования

№	Eps	Корень	Итерации
1	0.1	0.748691538955	2
2	0.01	0.747020993431	3
3	0.001	0.747116301166	4
4	0.0001	0.747116301166	4
5	0.00001	0.747110906543	5
6	0.000001	0.747111212029	6

Из таблицы видно, что при уменьшении значения *Eps* найденное приближение становится более точным и приближается к эталонному значению корня. При этом число итераций увеличивается.

Чувствительность метода простых итераций к ошибкам в исходных данных.

Для анализа чувствительности метода Ньютона к ошибкам в исходных данных код был дополнен функцией Round, которая округляла значения функции и производных с заданной точностью Delta. Это позволяло моделировать влияние ошибок исходных данных на результат вычислений.

Для каждого значения Eps перебиралось несколько значений Delta, после чего определялось приближённое значение корня методом Ньютона. Полученные результаты приведены в табл. 3.

Таблица 3 – Исследование чувствительности метода к ошибкам в данных

Eps	Delta	Результаты метода
0.1	0.1	0.700000000000
0.1	0.01	0.750000000000
0.1	0.001	0.749000000000
0.1	0.0001	0.748700000000

0.1	0.00001	0.7486900000000
0.1	0.000001	0.7486920000000
0.01	0.1	0.7000000000000
0.01	0.01	0.7500000000000
0.01	0.001	0.7470000000000
0.01	0.0001	0.7470000000000
0.01	0.00001	0.7470200000000
0.01	0.000001	0.7470210000000
0.001	0.1	0.7000000000000
0.001	0.01	0.7500000000000
0.001	0.001	0.7470000000000
0.001	0.0001	0.7471000000000
0.001	0.00001	0.7471200000000
0.001	0.000001	0.7471160000000
0.0001	0.1	0.7000000000000
0.0001	0.01	0.7500000000000
0.0001	0.001	0.7470000000000
0.0001	0.0001	0.7471000000000
0.0001	0.00001	0.7471200000000
0.0001	0.000001	0.7471160000000
0.00001	0.1	0.7000000000000
0.00001	0.01	0.7500000000000
0.00001	0.001	0.7470000000000
0.00001	0.0001	0.7471000000000
0.00001	0.00001	0.7471100000000
0.00001	0.000001	0.7471110000000
0.000001	0.1	0.7000000000000
0.000001	0.01	0.7500000000000
0.000001	0.001	0.7470000000000
0.000001	0.0001	0.7471000000000
0.000001	0.00001	0.7471100000000
0.000001	0.000001	0.7471110000000

Выводы.

В ходе выполнения лабораторной работы изучены методы бисекции, хорд, простых итераций и метод Ньютона, написана программа с использованием данных методов вычисляющая результат функции с различной точностью. Проведены исследование скорости сходимости и также обусловленности метода.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЕ КОДЫ ПРОГРАММ

Метод бисекции:

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

double c = 1.0;

// 2) Подпрограмма вычисления функции fx
double F(double x) {
    return (1.0 + cos(x)) / (3.0 - sin(x)) - x;
}

// 3) Подпрограмма вычисления производной f'x
double dF(double x) {
    double numerator =
        (-sin(x)) * (3.0 - sin(x)) -
        (1.0 + cos(x)) * (-cos(x));

    double denominator =
        (3.0 - sin(x)) * (3.0 - sin(x));

    return numerator / denominator - 1.0;
}

// Вспомогательная функция округления для исследования обусловленности
double Round(double X, double Delta) {
    if (Delta <= 0) return X;
    if (X > 0.0) return Delta * (long((X / Delta) + 0.5));
    else return Delta * (long((X / Delta) - 0.5));
}

// Метод бисекции с подсчетом итераций N
double BISECT(double Left, double Right, double Eps, int &N) {
    double FLeft = F(Left);
    double X;
    double Y;
    N = 0;

    // Условие выхода: длина отрезка стала меньше 2*Eps
    while ((Right - Left) >= 2.0 * Eps) {
        X = (Left + Right) / 2.0;
        Y = F(X);

        if (Y == 0.0) return X;

        if (FLeft * Y < 0)
            Right = X;
        else {
            Left = X;
            FLeft = Y;
        }
        N++;
    }
    return (Left + Right) / 2.0;
}
```

```

// Метод бисекции с округлением значений функции (для задания 6)
double BISECT_Rounded(double Left, double Right, double Eps, double
Delta, int &N) {
    double FLeft = Round(F(Left), Delta);
    double X;
    double Y;
    N = 0;

    while ((Right - Left) >= 2.0 * Eps) {
        X = (Left + Right) / 2.0;
        Y = Round(F(X), Delta);

        if (Y == 0.0) return X;

        if (FLeft * Y < 0)
            Right = X;
        else {
            Left = X;
            FLeft = Y;
        }
        N++;
    }
    return (Left + Right) / 2.0;
}

// 4) Головная программа
int main() {
    double Left = 0.0;
    double Right = 1.0;
    int N;
    double root;

    // Задание 5: Вычисления зависимости N от Eps
    printf("Исследование зависимости N от Eps (Задание 5):\n");

    printf("-----\n");
    printf("      Eps      |      Root x      |  N  |  f(x)      |  f'(x)      |
Delta_teor \n");
    printf("-----\n");

    // Исследуем диапазон точности от 1e-2 до 1e-12
    for (double Eps = 1e-2; Eps >= 1e-12; Eps /= 10.0) {
        root = BISECT(Left, Right, Eps, N);

        double fx = F(root);
        double dfx = dF(root);

        // Теоретическая оценка погрешности (обусловленность): |f(x) /
f'(x)|
        double delta_teor = fabs(fx / dfx);

        printf("%10.0e | %14.10f | %3d | %9.2e | %7.4f | %9.2e\n",
            Eps, root, N, fx, dfx, delta_teor);
    }

    printf("-----\n");

```

```

        // Вывод данных для графика зависимости N от Eps (в формате для
Python)
printf("\nДанные для графика (Eps, N):\n");
for (double Eps = 1e-2; Eps >= 1e-12; Eps /= 10.0) {
    root = BISECT(Left, Right, Eps, N);
    printf("Eps: %e, N: %d\n", Eps, N);
}

// Пример использования Round
double delta_err = 1e-4;
printf("\nПример работы с Round (Delta = %.1e):\n", delta_err);
double test_val = -0.88137358;
printf("Оригинал: %.8f -> Round: %.8f\n", test_val, Round(test_val,
delta_err));

// Задание 6: Исследование чувствительности к ошибкам
double Eps_fixed = 1e-12;
printf("\nИсследование чувствительности к ошибкам в вычислении
функции (Eps=%0e) (Задание 6):\n", Eps_fixed);

printf("-----\n");
printf("      Delta      |      Root x      |  N  |      actual f(root)
\n");

printf("-----\n");

for (double Delta = 1e-1; Delta >= 1e-8; Delta /= 10.0) {
    root = BISECT_Rounded(Left, Right, Eps_fixed, Delta, N);
    double actual_fx = F(root); // Истинное значение без
округления
    printf("%10.0e | %14.10f | %3d | %14.5e\n",
        Delta, root, N, actual_fx);
}
// Для Delta=0 (без округления)
root = BISECT(Left, Right, Eps_fixed, N);
double actual_fx = F(root);
printf("%10.0e | %14.10f | %3d | %14.5e\n",
    0.0, root, N, actual_fx);

printf("-----\n");

return 0;
}

```

Метод Хорд:

```

#include <stdio.h>
#include <math.h>
#include <stdlib.h>

double current_delta = 1e-10; // точность округления

// функция округления
double Round(double X, double Delta) {
    if (Delta <= 0) return X;

    if (X > 0.0)

```



```

        return Delta * (long)((X / Delta) + 0.5);
    else
        return Delta * (long)((X / Delta) - 0.5);
}

// точная функция
double F(double x) {
    return (1.0 + cos(x)) / (3.0 - sin(x)) - x;
}

// функция с округлением (для исследования ошибок)
double F_round(double x) {
    return Round(F(x), current_delta);
}

// производная
double dF(double x) {
    double numerator =
        (-sin(x)) * (3.0 - sin(x)) -
        (1.0 + cos(x)) * (-cos(x));

    double denominator =
        (3.0 - sin(x)) * (3.0 - sin(x));

    return numerator / denominator - 1.0;
}

// метод хорд
double HORDA(double Left, double Right, double Eps, int &N) {
    double FLeft = F(Left);
    double FRight = F(Right);

    double X, Y;

    if (FLeft * FRight > 0.0) {
        puts("Ошибка: Функция не меняет знак на интервале");
        exit(1);
    }

    N = 0;

    do {
        X = Left - (Right - Left) * FLeft / (FRight - FLeft);

        Y = F_round(X);

        if (Y * FLeft < 0.0) {
            Right = X;
            FRight = Y;
        }
        else {
            Left = X;
            FLeft = Y;
        }

        N++;
    } while (fabs(Y) >= Eps);
}

```

```

        return X;
    }

    // главная программа
    int main() {

        double Left = 0.0;
        double Right = 1.0;

        int N = 0;

        double root;

        double prev = 0.0;
        double prev2 = 0.0;

        printf("Исследование скорости сходимости метода хорд\n");

        printf("-----\n");
        printf("Итерации | Корень | Ошибка | C\n");
        printf("сходимости\n");
        printf("-----\n");

        for (int i = 1; i <= 20; i++) {

            double eps_iter = pow(10, -i);

            root = HORDA(Left, Right, eps_iter, N);

            double error = fabs(root - prev);

            double C = 0.0;

            if (i > 2) {

                double denom = fabs(prev - prev2);

                if (denom > 1e-15)
                    C = error / denom;
            }

            printf("%3d | %14.10f | %12.3e | %10.5f\n",
                i, root, error, C);

            prev2 = prev;
            prev = root;
        }

        printf("-----\n");

        printf("\nРезультаты вычислений:\n");
        printf("Корень уравнения x = %.10f\n", root);
        printf("f(x) = %.10e\n", F(root));
        printf("f'(x) = %.10f\n", dF(root));

        double reference = root;

        printf("\n\nТаблица 2 - Исследование чувствительности метода\n");
    }

```

```

printf("-----\n");

    for (int i = 1; i <= 6; i++)
    {
        double eps = pow(10, -i);

        printf("\nEps = 1e-%d\n", i);
        printf("Delta      Root      Error\n");

        for (int j = 1; j <= 6; j++)
        {
            double Left = 0.0;
            double Right = 1.0;

            current_delta = pow(10, -j);

            int iter;

            double r = HORDA(Left, Right, eps, iter);

            double error = fabs(r - reference);

            printf("1e-%d   %12.10f   %10.3e\n",
                    j, r, error);
        }
    }

printf("\n-----\n");

    return 0;
}

```

Метод Ньютона:

```

#include <iostream>
#include <math.h>
#include <iomanip>
#include <vector>

double Round (double X,double Delta)
{
    if (Delta<=1E-9) {puts("Неверное задание точности
округления\n");exit(1);}
    if (X>0.0) return (Delta*(long((X/Delta)+0.5)));
    else return (Delta*(long((X/Delta)-0.5)));
}

double F(double x) {
    return (1.0 + cos(x)) / (3.0 - sin(x)) - x;
}

double F_round(double x, double delta) {
    return Round(F(x), delta);
}

double dF(double x){

```

```

        double numerator = (1.0 - 3 * sin(x) + cos(x));
        double denominator = (3 - sin(x)) * (3 - sin(x));

        return numerator / denominator - 1;
    }

double ddF(double x){
    double numerator = ((-3*cos(x) - sin(x)) * (3 - sin(x)) * (3 -
sin(x))) + (2 * (1 - 3 * sin(x) + cos(x)) * (3 - sin(x)) * cos(x));
    double denominator = (3 - sin(x)) * (3 - sin(x)) * (3 - sin(x)) *
(3 - sin(x));

    return numerator / denominator;
}

double NEWTON (double X,double Eps,int &N)
{
    extern double F1 (double);
    double Y,Y1,DX;
    N=0;
    do
    {
        Y = F(X);
        if (Y==0.0) return (X);

        Y1 = dF(X);
        if (Y1==0.0) {puts("Производная обратилась в ноль\n");exit(1);}

        DX=Y/Y1; X=X-DX; N++;
    }
    while (fabs(DX)>Eps);
    return (X);
}

int main(){
    double Left = 0.0;
    double Right = 1.0;
    double x0 = 1.0;
    int N = 0;

    std::cout << std::fixed << std::setprecision(12);

    std::cout << "Отрезок: [" << Left << ", " << Right << "]\n";
    std::cout << "Начальное приближение x0 = " << x0 << "\n\n";

    // Эталонный корень
    std::vector<double> steps_true;
    double X = x0;
    double DX;

    do {
        double Y = F(X);
        double Y1 = dF(X);
        DX = Y / Y1;
        X = X - DX;
        steps_true.push_back(X);
    } while (fabs(DX) > 1e-15);
}

```

```

double root_true = X;

std::cout << "Эталонный корень: " << root_true << "\n\n";

// Тестирование точности
std::cout << "Eps\t\tКорень\n";

for (int i = 1; i <= 6; i++) {
    double eps = pow(10.0, -i);

    double Xtest = x0;
    double DXtest;

    do {
        double Y = F(Xtest);
        double Y1 = dF(Xtest);
        DXtest = Y / Y1;
        Xtest = Xtest - DXtest;
    } while (fabs(DXtest) > eps);

    std::cout << eps << "\t" << Xtest << "\n";
}

// Скорость сходимости
std::cout << "\nСкорость сходимости:\n";
std::cout << "n\t x_n\t\t\tОшибка\n";

for (size_t i = 0; i < steps_true.size(); i++) {
    double err = fabs(steps_true[i] - root_true);
    std::cout << i+1 << "\t" << steps_true[i] << "\t" << err <<
"\n";
}

// Чувствительность (Delta)
std::cout << "\nЧувствительность:\n";

for (int i = 1; i <= 6; i++) {
    double eps = pow(10.0, -i);
    std::cout << "\nEps = " << eps << "\n";
    std::cout << "Delta\t\tКорень\n";

    for (int j = 1; j <= 6; j++) {
        double delta = pow(10.0, -j);

        double Xr = x0;
        double DXr;

        do {
            double Y = Round(F(Xr), delta);
            double Y1 = Round(dF(Xr), delta);

            DXr = Y / Y1;
            Xr = Xr - DXr;

        } while (fabs(Round(F(Xr), delta)) > eps);

        std::cout << delta << "\t" << Xr << "\n";
    }
}

```

```

    }
}

return 0;
}

```

Метод простых итераций:

```

#include <iostream>
#include <math.h>
#include <iomanip>
#include <vector>

double Round (double X,double Delta)
{
    if (Delta<=1E-9) {puts("Неверное задание точности
округления\n");exit(1);}
    if (X>0.0) return (Delta*(long((X/Delta)+0.5)));
    else      return (Delta*(long((X/Delta)-0.5)));
}

double F(double x) {
    return (1.0 + cos(x)) / (3.0 - sin(x)) - x;
}

double phi(double x) {
    return (1.0 + cos(x)) / (3.0 - sin(x));
}

double dphi(double x){
    return (1.0 - 3.0 * sin(x) + cos(x)) / pow((3.0 - sin(x)), 2);
}

double phi_round(double x, double delta) {
    return Round(phi(x), delta);
}

// Универсальная функция итераций
double iter(double x0, double eps, std::vector<double>& steps, int&
iterations) {
    if (eps <= 0.0) {
        std::cout << "Точность должна быть положительной.\n";
        return NAN;
    }

    double x = x0;
    iterations = 0;

    for (int i = 0; i < 10000; i++) {
        steps.push_back(x);
        iterations++;

        double x_next = phi(x);

        if (std::fabs(x_next - x) < eps) {
            steps.push_back(x_next);

```

```

        return x_next;
    }

    x = x_next;
}

std::cout << "Достигнуто максимальное количество итераций.\n";
return x;
}

double iter_round(double x0, double eps, double delta, int& iterations)
{
    if (eps <= 0.0) {
        return NAN;
    }

    double x = x0;
    iterations = 0;

    for (int i = 0; i < 10000; i++) {
        iterations++;

        double x_next = phi_round(x, delta);

        if (std::fabs(x_next - x) < eps) {
            return x_next;
        }

        x = x_next;
    }

    return x;
}

int main(){
    double Left = 0.0;
    double Right = 1.0;
    double x0 = 1.0;

    std::cout << std::fixed << std::setprecision(12);

    // Проверка отрезка локализации
    std::cout << "Проверка отрезка локализации:\n";
    std::cout << "F(" << Left << ") = " << F(Left) << "\n";
    std::cout << "F(" << Right << ") = " << F(Right) << "\n\n";

    // Проверка условия сходимости
    std::cout << "Проверка условия сходимости метода простых
итераций:\n";
    std::cout << "phi'(Left) = " << dphi(Left) << "\n";
    std::cout << "phi'(Right) = " << dphi(Right) << "\n";
    std::cout << "|phi'(Left)| = " << fabs(dphi(Left)) << "\n";
    std::cout << "|phi'(Right)| = " << fabs(dphi(Right)) << "\n\n";

    // Эталонный корень
    std::vector<double> steps_true;
    int N_true = 0;

```

```

double root_true = iter(x0, 1e-15, steps_true, N_true);

std::cout << "Эталонный корень: " << root_true << "\n";
std::cout << "Итерации: " << N_true << "\n\n";

// Тестирование
std::cout << "Тестирование:\n";
std::cout << "Eps\t\t\tКорень\t\t\tИтерации\n";

for (int i = 1; i <= 6; i++) {
    double eps = pow(10.0, -i);
    std::vector<double> steps_test;
    int N_test = 0;
    double root_test = iter(x0, eps, steps_test, N_test);
    std::cout << eps << "\t" << root_test << "\t" << N_test <<
"\n";
}

// Исследование скорости сходимости
std::cout << "\nИсследование скорости сходимости:\n";
std::vector<double> steps;
int it_conv = 0;
iter(x0, 1e-10, steps, it_conv);

std::cout << "Количество итераций: " << it_conv << "\n";
std::cout << "n\t x_n\t\t\tОшибка\n";

for (size_t i = 0; i < steps.size(); i++) {
    double err = std::fabs(steps[i] - root_true);
    std::cout << i + 1 << "\t" << steps[i] << "\t" << err << "\n";
}

// Исследование чувствительности
std::cout << "\nИсследование чувствительности:\n";

for (int i = 1; i <= 6; i++) {
    double eps = std::pow(10.0, -i);
    std::cout << "\nEps = " << eps << "\n";
    std::cout << "Delta\t\t\tКорень\t\t\tИтерации\n";

    for (int j = 1; j <= 6; j++) {
        double delta = std::pow(10.0, -j);
        int it = 0;
        double root = iter_round(x0, eps, delta, it);

        std::cout << delta << "\t" << root << "\t" << it << "\n";
    }
}

return 0;
}

```